

Object-Oriented Middleware (RMI & CORBA)

Middleware Architecture (SCS 2314)

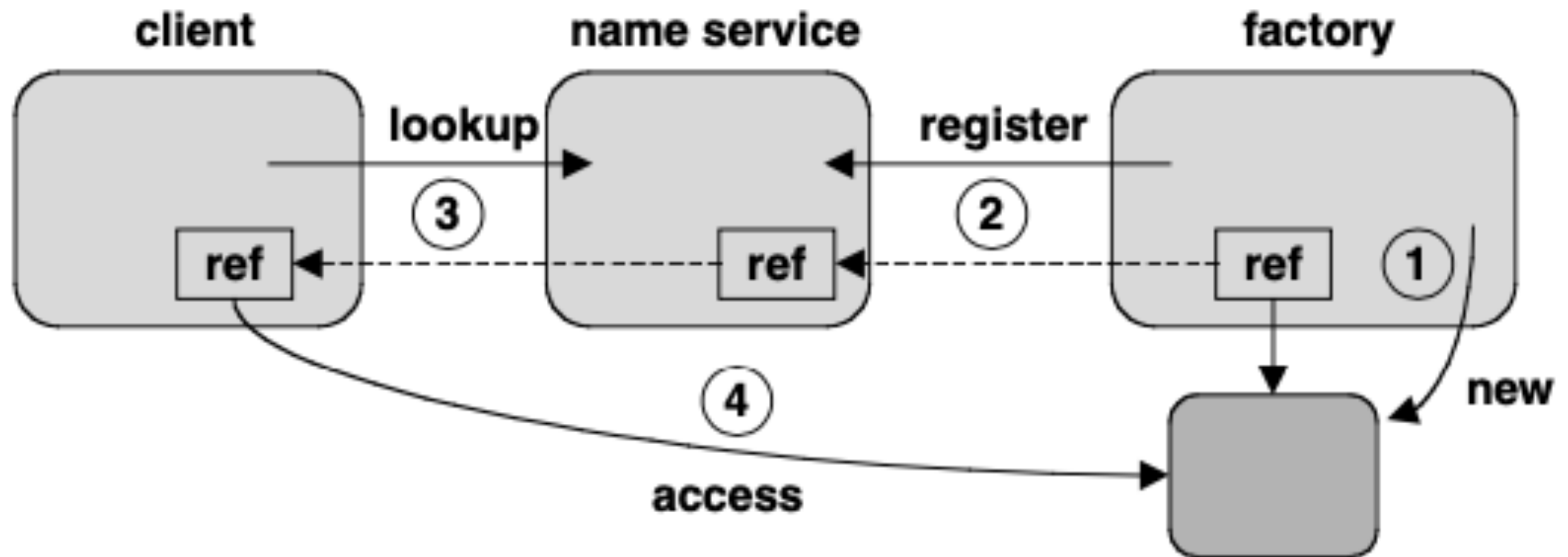
Distributed Object Systems

- Distributed object systems are **distributed** systems in which all entities are **modelled as objects**: Related to the **Object-Oriented Paradigm**.
- **Distributed object systems** are a popular paradigm for **object-oriented distributed applications**.
- Since the application is modelled as a set of **cooperating objects**, it maps very naturally to the services of the distributed system.

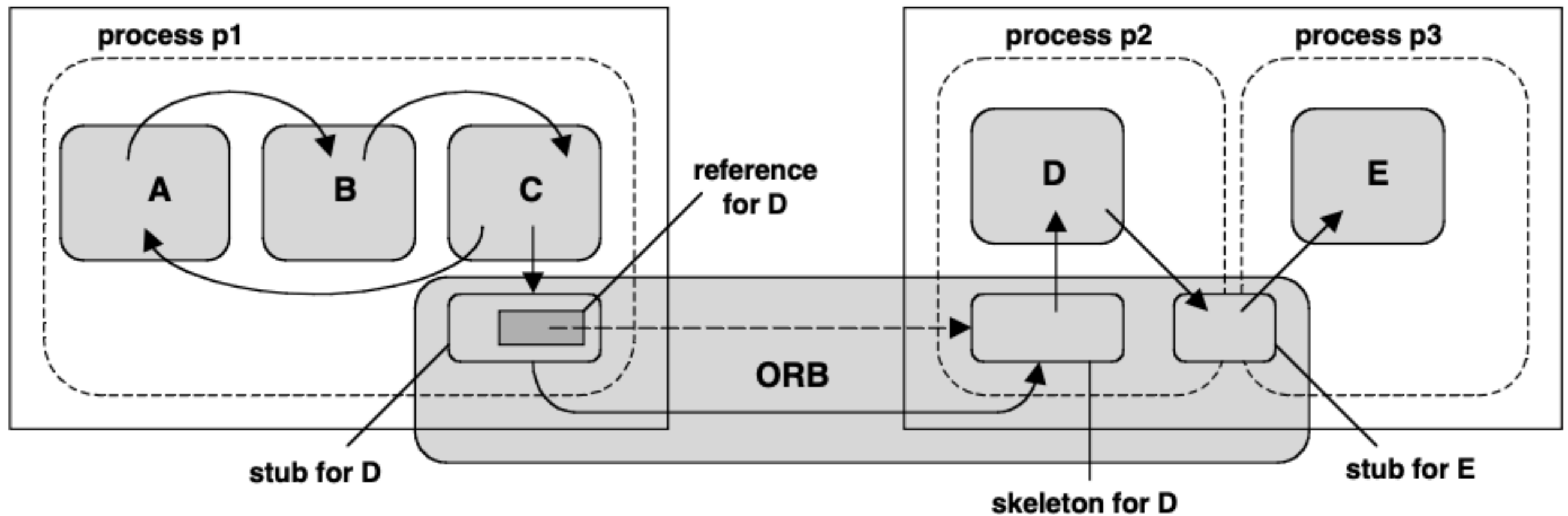
Benefits - Distributed Object Systems

- Application designers may take **advantage of the expressiveness, abstraction, and flexibility** of an object model.
- **Encapsulation** allows an object's implementation to be placed on any site; object placement may be done according to specific criteria such as access locality, administration constraints, security, etc.
- **Legacy applications** may be reused by **encapsulating** them in objects.
- **Scalability** is enhanced by distributing processing power over a network of servers, which may be extended to accommodate an increasing load.

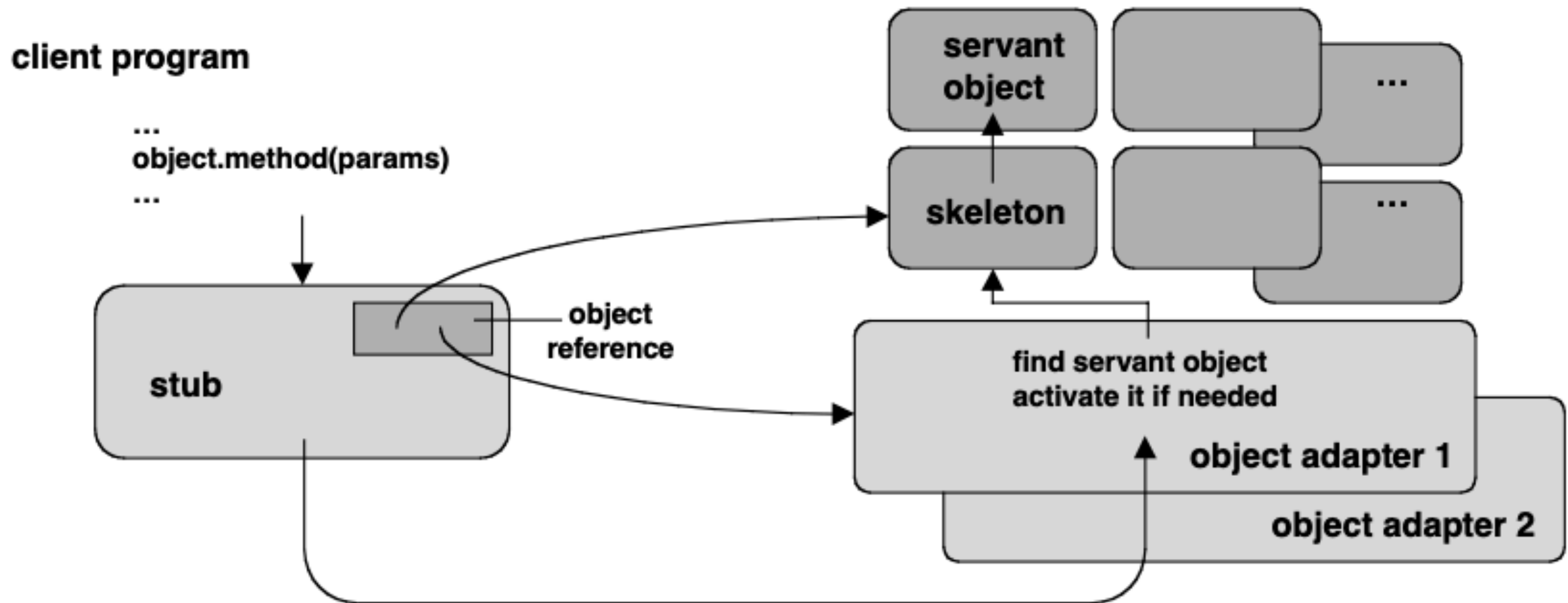
Server Object Creation - Factory



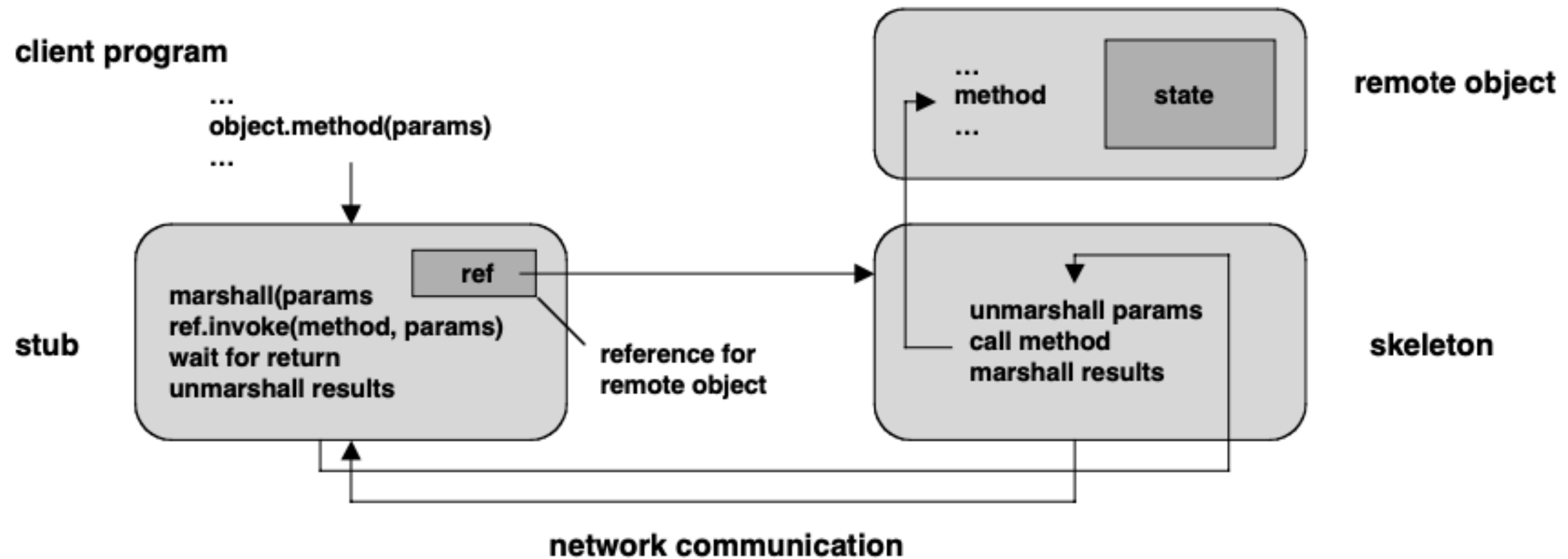
Remote and Local Invocation



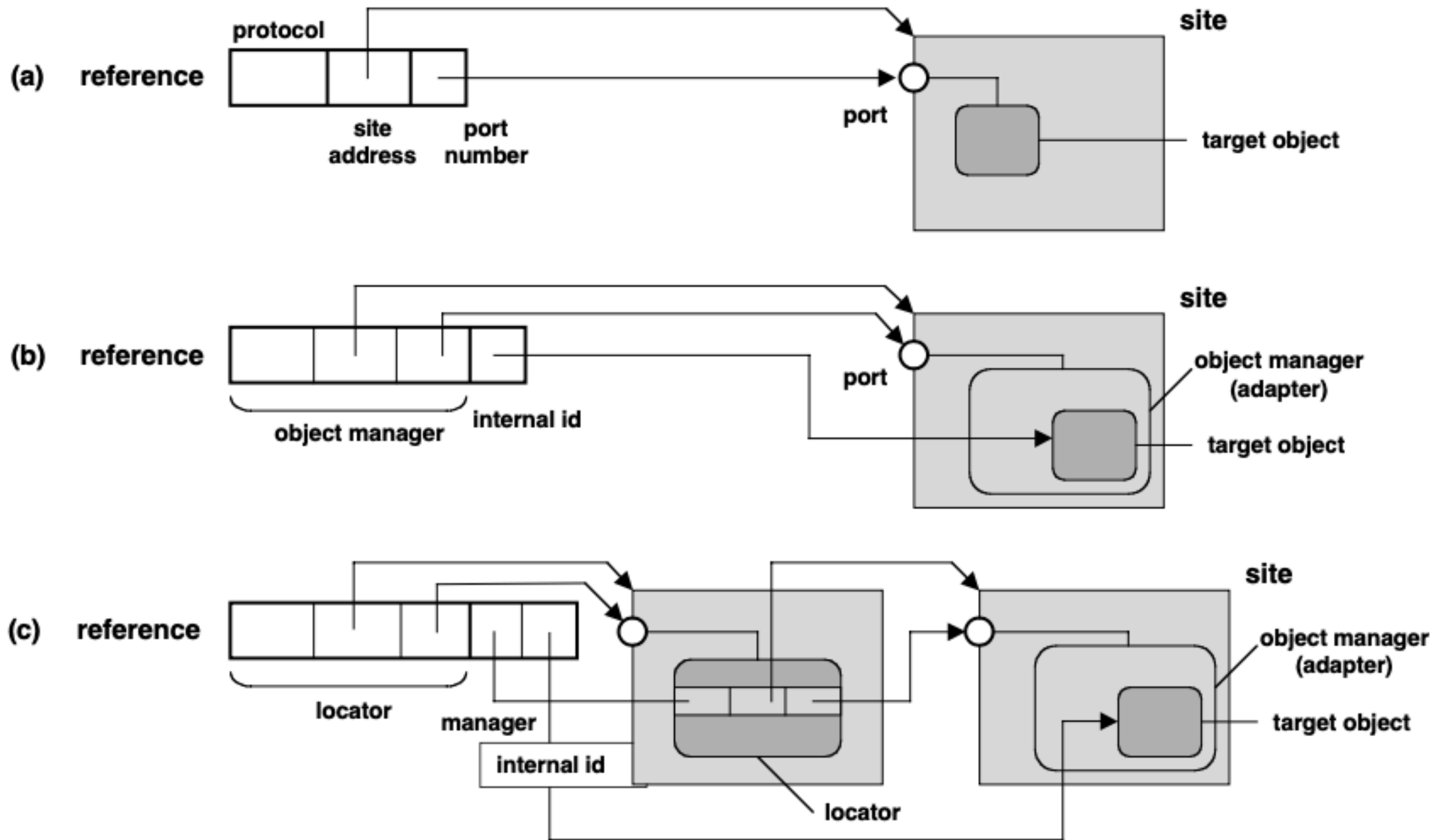
Invoking Remote Object through an Adaptor



Remote Invocation Details



Remote Invocation Details

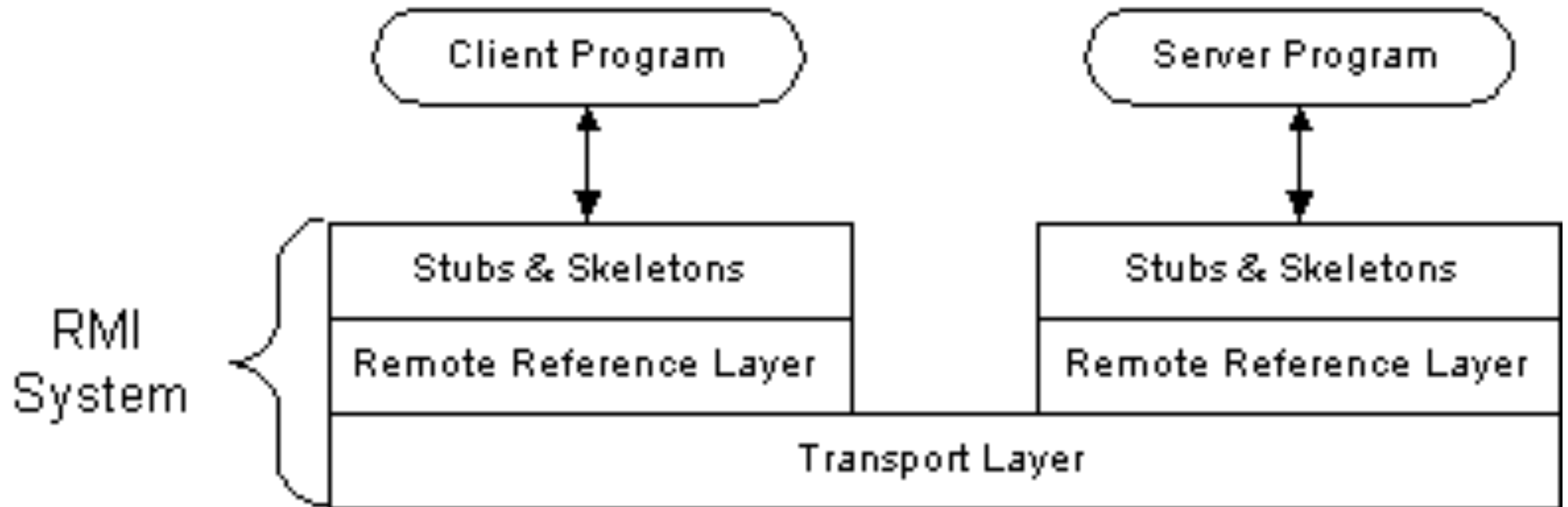


Remote Method Invocation

RMI Design Goals

The **design goal** for the RMI architecture was to create a **Java distributed object model** that **integrates** naturally into Java language and **local object model**.

RMI Architecture



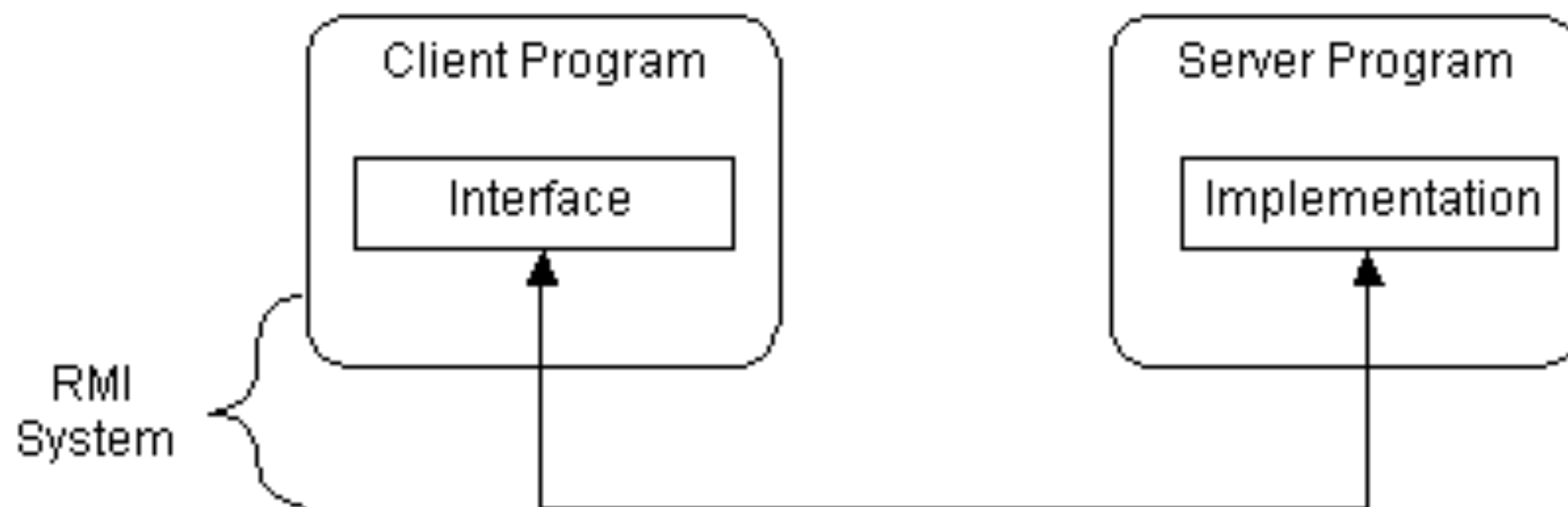
RMI Architecture

The RMI implementation is essentially built from three abstraction layers.

- The first is the **Stub and Skeleton layer**, which lies just beneath the view of the developer. This layer intercepts method calls made by the client to the interface reference variable and redirects these calls to a remote RMI service.
- The next layer is the **Remote Reference Layer**. This layer understands how to interpret and manage references made by clients to remote service objects.
- The **Transport layer** is based on TCP/IP connections between machines in a network. It provides basic connectivity, as well as some firewall penetration strategies.

RMI Architecture

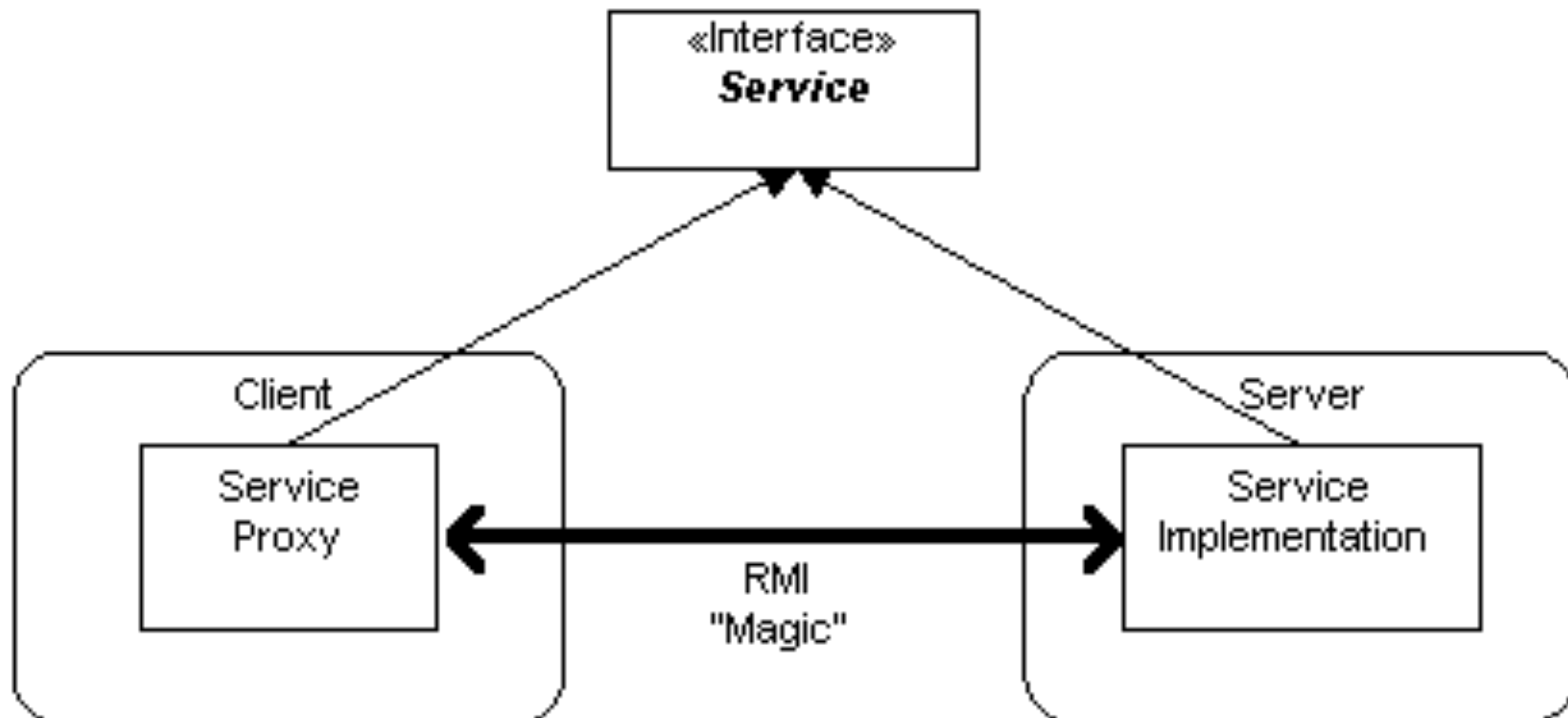
- In RMI, the definition of a remote service is coded using a Java **interface**.
- The implementation of the remote service is coded in a **class**.
- The key to understanding RMI is to remember that interfaces define **behaviour** and classes define the **implementation**.



RMI Architecture

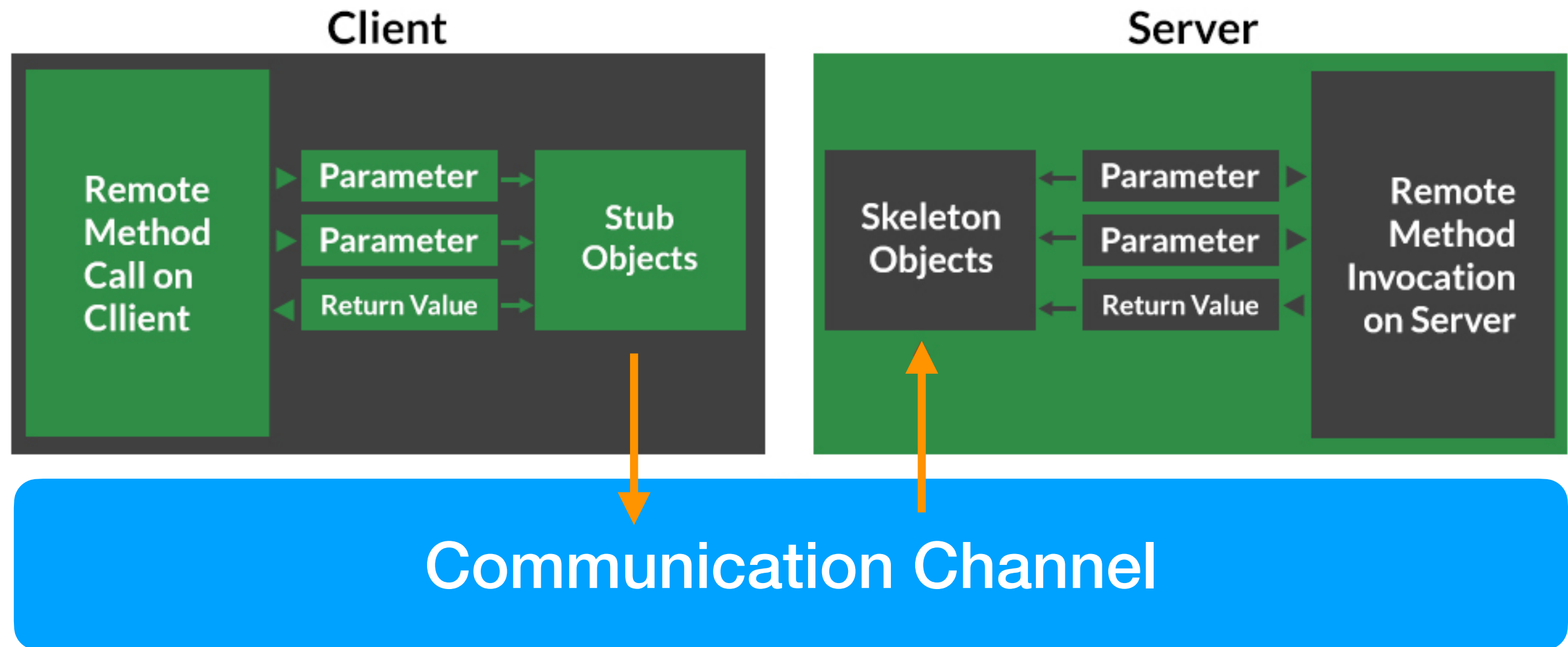
RMI supports two classes that implement the same interface.

- The first class is the implementation of the behaviour, and it runs on the server.
- The second class acts as a proxy for the remote service and it runs on the client.



RMI Architecture

Working of RMI



Remote Naming Service

- Clients find remote services by using a naming or directory service. A naming or directory service is run on a well-known host and port number.
- RMI can use many different directory services, including the **Java Naming and Directory Interface (JNDI)**.
- RMI itself includes a simple service called the RMI Registry, **“rmiregistry”**.
- The RMI Registry runs on each machine that hosts remote service objects and accepts queries for services, by default on port 1099.

RMI Example

Local Method Invocation - Example

```
// Hello Interface
public interface Hello {
    String sayHello();
};

// Hello Implementation
class HelloImpl implements Hello {
    HelloImpl() {    // constructor
    };
    public String sayHello() {
        return "Hello World!";
    };
}

// Hello Usage
...
Hello hello = new HelloImpl ();
hello.sayHello();
```

Remote Method Invocation - Example

```
public interface Hello extends Remote {  
    String sayHello() throws RemoteException;  
}
```

Interface-Proxy

```
class HelloImpl extends UnicastRemoteObject implements Hello {  
    HelloImpl() throws RemoteException {  
    };  
    public String sayHello() throws RemoteException {  
        return "Hello World!";  
    };  
}
```

**Interface-
Implementation**

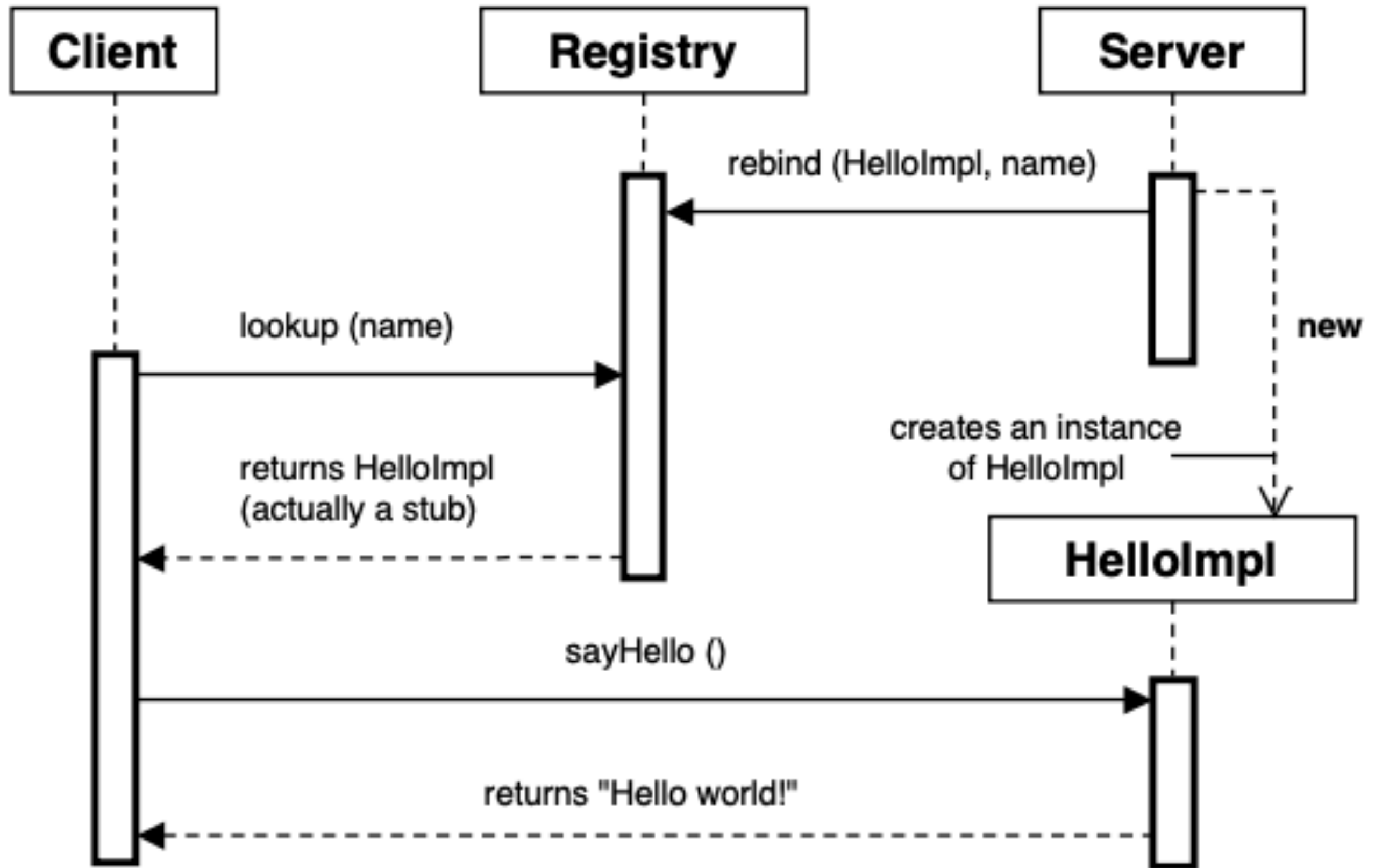
```
public class Server {  
    public static void main (...) {  
        ...  
        Naming.rebind("jrmii:/// " + registryHost + "/helloobj", new HelloImpl());  
        System.out.println("Hello Server ready !");  
    }  
}
```

**Remote Object-
Creation and
Registration**

```
...  
Hello obj = (Hello) Naming.lookup("jrmii:/// " + registryHost + "/helloobj");  
System.out.println(obj.sayHello());
```

**Client-
Look-up and
Remote Call**

Remote Method Invocation - Example



CORBA

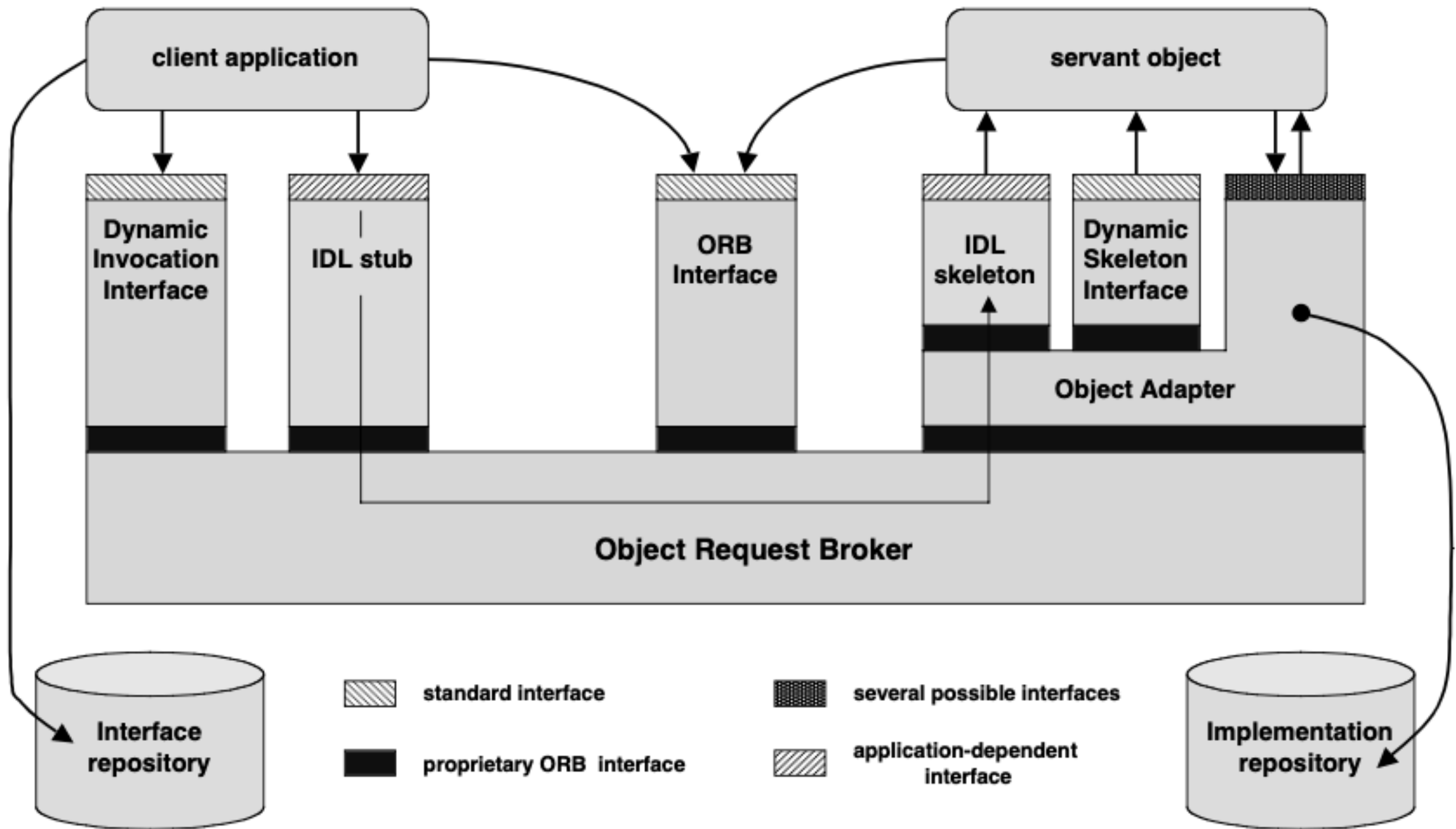
C O R B A

- **CORBA (Common Object Request Broker Architecture)**, is a standard architecture for distributed object systems.
- CORBA allows a **distributed, heterogeneous** collection of objects to **inter-operate**.
- CORBA products provide a **framework** for the development and execution of distributed applications.

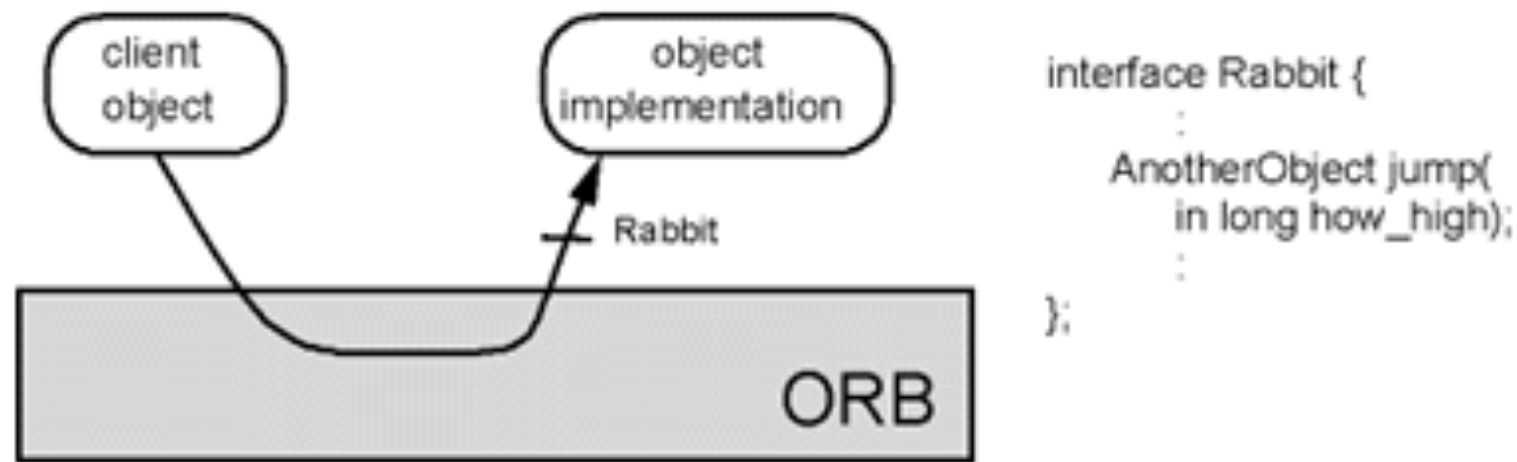
OMG

- The Object Management Group (OMG) is responsible for defining CORBA.
- The OMG comprises over 700 companies and organisations, including almost all the major vendors and developers of distributed object technology, including platform, database, and application vendors as well as software tool and corporate developers.

CORBA Architecture



CORBA Architecture



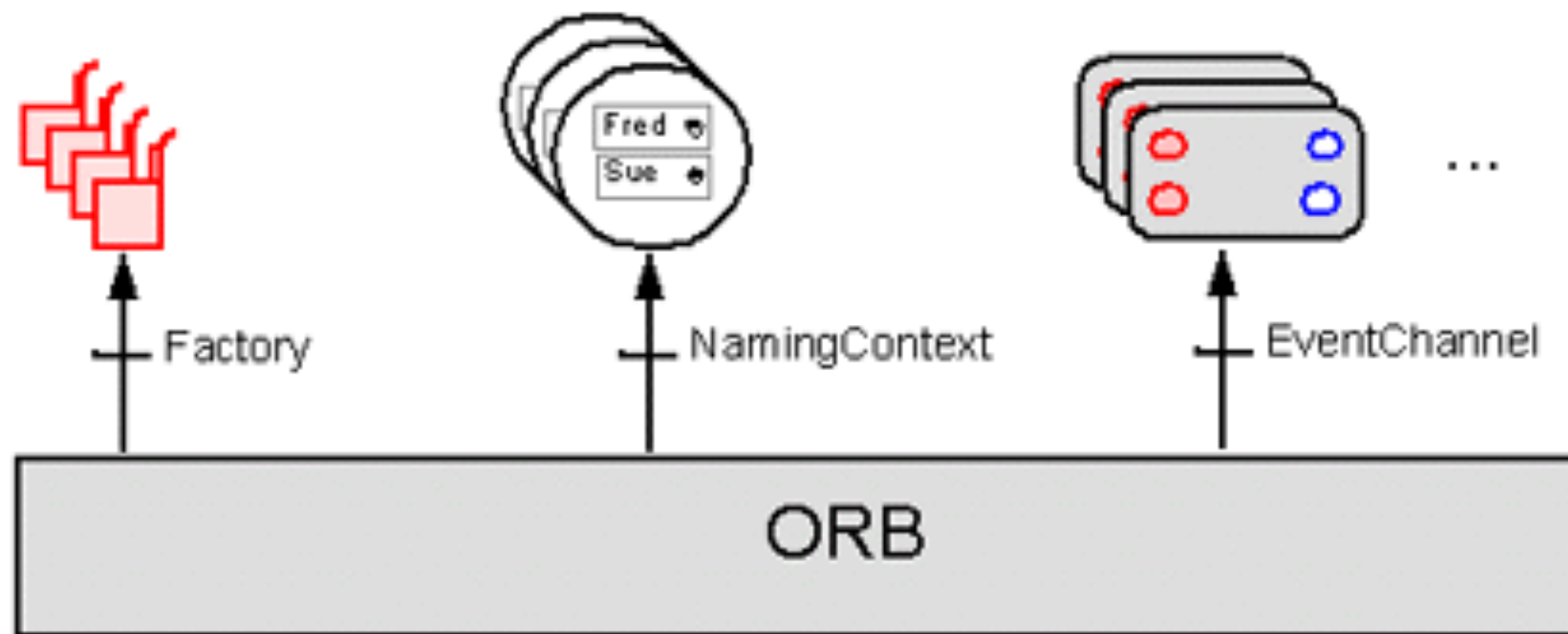
- A **client** holds an object **reference** to a distributed object.
- The **object reference** is **typed** by an **interface** (Rabbit).
- The Object Request Broker, or ORB, **delivers** the request to the object and **returns** any **results** to the **client**.
- E.g. A jump request returns an object reference typed by the AnotherObject interface.

Object Request Broker (ORB)

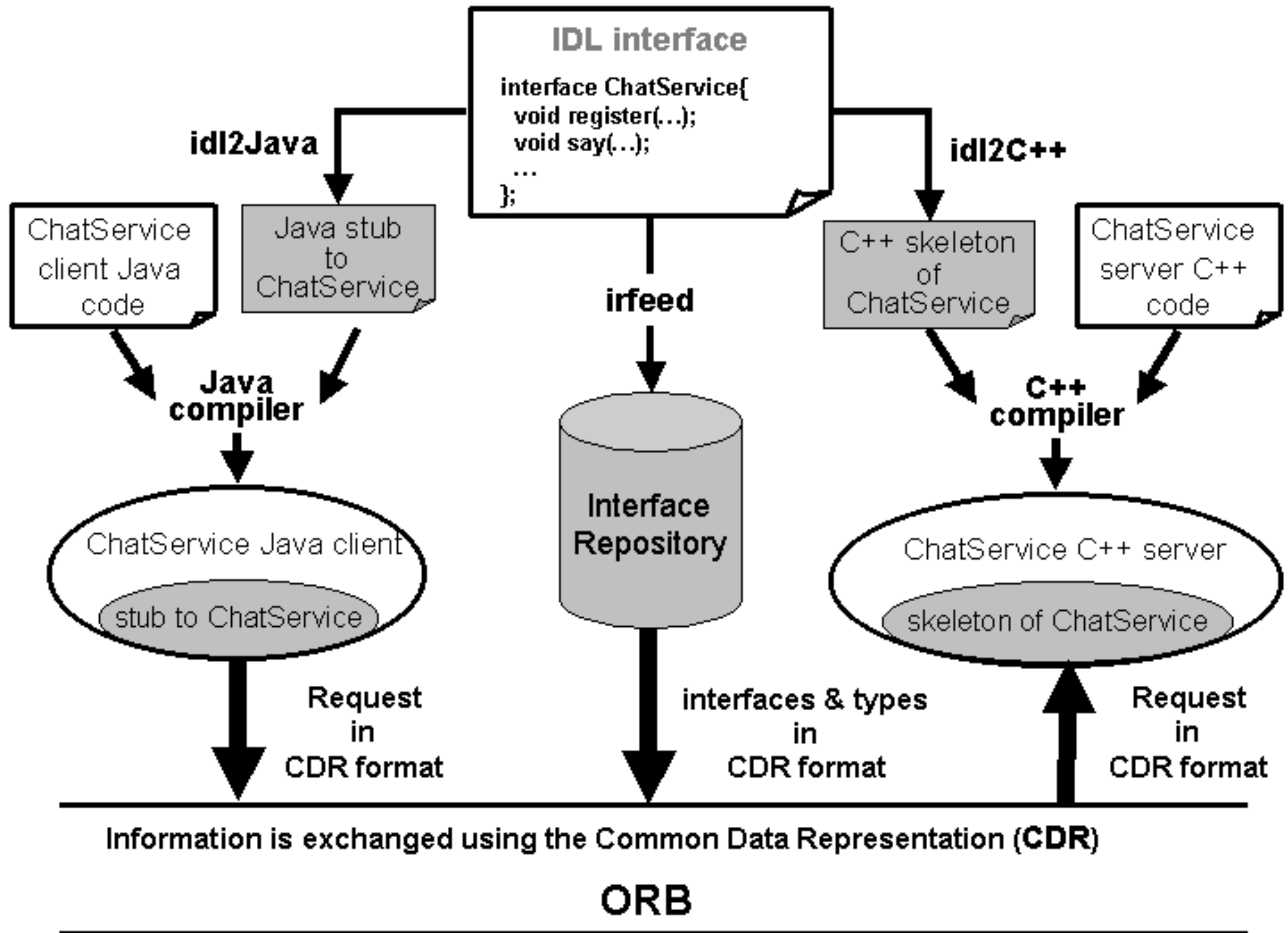
- The ORB is the **distributed service** that implements the request to the remote object.
- It **locates** the **remote object** on the network, **communicates** the request to the object, **waits** for the **results** and when available **communicates** those results back to the **client**.
- The ORB implements **location transparency**.
- The ORB implements **programming language independence** for the request.

CORBA Services

CORBA services support the integration and interoperation of distributed objects.



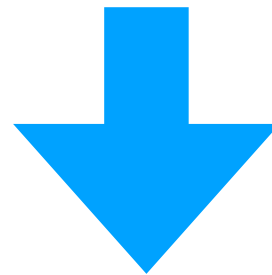
CORBA Architecture



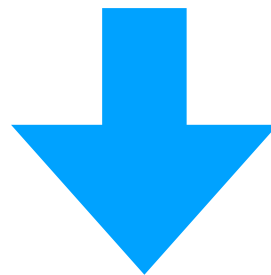
CORBA Example

CORBA Application-Interface

**The Interface Description is written in Language Neutral
Interface Definition Language (IDL)**



```
interface Hello {  
    string sayHello();  
};
```



**There are tools to convert the IDL interface to
Language-specific interfaces.**

E.g. IDLtoJava converts IDL interface to Java Interface classes

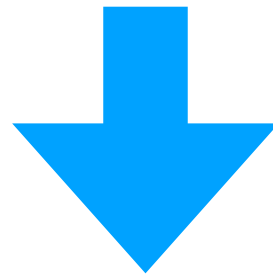
Generate the stub and skeleton files (HelloStub.java and HelloImplBase.java)

CORBA Application-Implementation

The generated stub and skeleton files

(HelloStub.java and HelloImplBase.java)

are used to implement the Language Specific Object Implementation



```
class HelloImpl extends _HelloImplBase {  
  
    HelloImpl() {}  
    public String sayHello() {  
        return "Hello World!";  
    }  
}
```

CORBA Application-Factory & Register Implementation

```
public class Server {
    public static void main (String[] args) {
        try {
            ORB orb = ORB.init(args,null);
            Hello hello = new HelloImpl();
            orb.connect(hello);
            IORHelpers.writeIORToFile(orb.object_to_string(hello),"hello_ior");
            org.omg.CORBA.Object ns_ref =
                orb.resolve_initial_references("NameService");
            NamingContext ns = NamingContextHelper.narrow(ns_ref);
            ns.rebind(new NameComponent[] { new NameComponent("helloobj","") },hello);
            System.out.println("Hello Server ready");
            orb.run();
        } catch (Exception e) {
            System.err.println("Hello Server exception");
            e.printStackTrace();
        }
    }
}
```

CORBA Application-Client Look-up and Call Remote Object

```
public class Client {
    public static void main(String[] args) {
        try {
            ORB orb = ORB.init(args,null);
            org.omg.CORBA.Object ns_ref =
                orb.resolve_initial_references("NameService");
            NamingContext ns = NamingContextHelper.narrow(ns_ref);
            org.omg.CORBA.Object obj_ref =
                ns.resolve(new NameComponent[] { new NameComponent("helloobj","") });
            Hello obj = HelloHelper.narrow(obj_ref);
            System.out.println(obj.sayHello());
        } catch (Exception e) {
            System.err.println("Hello Client exception");
            e.printStackTrace();
        }
    }
}
```

{ e n d }

“Well, you blink 14,000 times a day-so many that your eyes are shut for 23 minutes of every waking day.”

The Body: A guide for Occupants